

Komputerowo zintegrowane wytwarzanie

Temat 4

Opracowanie i implementacja algorytmów dla problemu $FP||C_{\max}$ lub $J||C_{\max}$

prowadzący: mgr inż. Piotr Nowak

1 Cel laboratorium

Celem laboratorium jest zapoznanie się metodami przybliżonymi na przykładzie problemów wielomaszynowych. Obejmuje to odpowiednie zdefiniowanie problemu (ograniczeń i funkcji celu), danych wejściowych oraz implementacje dedykowanych algorytmów, a także interpretację wyników.

2 Przebieg zajęć

Laboratorium obejmuje zajęcia nr 10-11 (4 godzin zajęć, w tym oddanie). Praca odbywa się w ramach dwuosobowych zespołów. Każdy zespół otrzymuje do opracowania problem do wyboru. W tym wypadku $FP||C_{\max}$ lub $J||C_{\max}$ oraz dedykowane dla niego algorytmy.

Zajęcia 10

1. Implementacja generacji instancji problemów dla zadanych parametrów (źródło, rozmiar, zakres).
2. Implementacja metody oceniania rozwiązania.
3. Implementacja algorytmu konstrukcyjnego.

Zajęcia 11

1. Dokończenie rozpoczętych implementacji.
2. Ocena efektów końcowych.

Ocenianie

- Ocena 3 - Algorytm konstrukcyjny NEH dla problemu $FP||C_{\max}$.
- Ocena 4 - Algorytm konstrukcyjny NEH dla problemu $FP||C_{\max}$ z 2 wybranymi modyfikacjami.
- Ocena 5 - Algorytm konstrukcyjny INSA dla problemu $J||C_{\max}$.

3 Problem

Permutacyjny problem przepływowy

Problem $FP||C_{\max}$ jest szczególnym przypadkiem problemu ogólniejszego $F||C_{\max}$, gdzie na każdej maszynie będziemy mieli taką samą kolejność wykonywania zadań. Mamy zbiór n zadań:

$$\mathcal{J} = \{1, 2, \dots, n\}, \quad (1)$$

które należy wykonać na m maszynach:

$$\mathcal{M} = \{1, 2, \dots, m\}, \quad (2)$$

każde j -te zadanie składa się dokładnie z m operacji

$$\mathcal{O}_j = \{o_{1,j}, o_{2,j}, \dots, o_{m,j}\}. \quad (3)$$

Każda operacja $o_{i,j}$ z zadania j jest wykonywana kolejno na następnej maszynie i według kolejności technologicznej w tym wypadku $1 \rightarrow 2 \rightarrow \dots \rightarrow m$. Czas wykonania (*performed time*) operacji $o_{i,j}$ wynosi $p_{i,j}$. Na każdej maszynie naraz może wykonywać się tylko jedna operacja. W problemie przepływowym w ramach zadań musi być zachowany porządek technologiczny, tzn. aby mogła się zacząć wykonywać kolejna operacja, najpierw musi się wykonać operacja poprzednia z tego samego zadania. Przez π oznaczmy kolejność wykonywania zadań (w permutacyjnym problemie przepływowym na każdej maszynie mamy tę samą kolejność wykonywania zadań).

W celu utworzenia harmonogramu dla zadanej permutacji π musi utworzyć macierz S momentów rozpoczęcia operacji oraz macierz C momentów zakończenia operacji.

$$S = \begin{bmatrix} S_{1,1} & S_{1,2} & S_{1,3} & \dots & S_{1,n} \\ S_{2,1} & S_{2,2} & S_{2,3} & \dots & S_{2,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ S_{m,1} & S_{m,2} & S_{m,3} & \dots & S_{m,n} \end{bmatrix} \quad (4)$$

$$C = \begin{bmatrix} C_{1,1} & C_{1,2} & C_{1,3} & \dots & C_{1,n} \\ C_{2,1} & C_{2,2} & C_{2,3} & \dots & C_{2,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ C_{m,1} & C_{m,2} & C_{m,3} & \dots & C_{m,n} \end{bmatrix} \quad (5)$$

Przy układaniu harmonogramu należy pamiętać o dwóch ograniczeniach. Po pierwsze, że aby mogła się zacząć wykonywać operacja aktualnego zadania na tej samej maszynie, najpierw musi się zakończyć operacja z zadania poprzedniego:

$$S_{i,\pi(j+1)} \geq C_{i,\pi(j)} \quad (6)$$

Po drugie, musi być zachowany porządek technologiczny, więc aby móc rozpocząć wykonywanie aktualnej operacji z zadania, to musi się zakończyć wykonywanie poprzedniej operacji w ramach tego samego zadania na poprzedniej maszynie:

$$S_{i+1,\pi(j)} \geq C_{i,\pi(j)} \quad (7)$$

Uwzględniając oba ograniczenia możemy wyznaczyć czas rozpoczęcia operacji wzorem:

$$S_{i,\pi(j)} = \max\{C_{i-1,\pi(j)}, C_{i,\pi(j-1)}\}. \quad (8)$$

Rozpatrywanym kryterium optymalizacyjnym jest czas zakończenia wykonywania wszystkich zadań $C_{\max}$

$$C_{\max} = \max_{j \in \mathcal{J}} C_{m,\pi(j)}. \quad (9)$$

Problem gniazdowy

Podobnie jak w problemie przepływowym, w problemie $J||C_{\max}$ mamy zbiór n zadań $\mathcal{J}$ oraz zbiór m maszyn $\mathcal{M}$. Najważniejsze różnice:

- Każde zadanie może mieć inną liczbę operacji.
- Każde zadanie może mieć inną marszrutę technologiczną, więc może zaczynać się na dowolnej maszynie, a także część maszyn może zostać pominięta.
- W ramach jednego zadania różne operacje mogą wykonywać się na tej samej maszynie, więc liczba operacji może być większa niż liczba maszyn.

Podsumowując, każde zadanie składa się z o_j operacji dla $j \in \mathcal{J}$. Dla ułatwienia możemy zmienić numerację operacji, numerując kolejno wszystkie operacje ze wszystkich zadań. Wtedy zbiór wszystkich operacji można zapisać następująco:

$$\mathcal{O} = \{1, 2, \dots, o_j, o_j + 1, \dots, \sum_{k \in \mathcal{J}} o_k\} \quad (10)$$

Do rozwiązania problemu można użyć dwóch reprezentacji.

Krotka permutacji

W zależności od reprezentacji dane wejściowe będą się trochę różnić:

- czas wykonywania i -tej operacji w zadaniu j -tym wynosi $p_{i,j}$,
- i -tą operację w zadaniu j -tym należy wykonać na maszynie $\mu_{i,j}$

Rozwiązaniem będzie krotka permutacji $\pi = (\pi_1, \pi_2, \dots, \pi_m)$, gdzie π_i to kolejność wykonywania zadań na i -tej maszynie. W celu utworzenia harmonogramu dla zadanej permutacji π należy wyznaczyć momenty rozpoczęcia wykonywania oraz momenty zakończenia wykonywania operacji. Należy pamiętać, że na wszystkich maszynach może być różna liczba operacji.

Jedna permutacja

W zależności od reprezentacji dane wejściowe będą się trochę różnić:

- czas wykonywania k -tej operacji wynosi ρ_k ,
- k -tą operację należy wykonać na maszynie η_k ,

Rozwiązaniem będzie jedna kolejność wykonywania wszystkich operacji π o długości równej liczbie operacji $|\mathcal{O}|$. W celu utworzenia harmonogramu dla zadanej permutacji π należy wyznaczyć momenty rozpoczęcia wykonywania oraz momenty zakończenia wykonywania operacji. Należy uważać, ponieważ nie wszystkie rozwiązania są dopuszczalne.

4 Metody rozwiązania

Algorytm NEH

Algorytm o charakterze konstrukcyjnym dla $FP||C_{\max}$ opiera się na metodzie wstawiania co-raz to mniejszych zadań konsekwentnie na wszystkich możliwych pozycjach. Należy zacząć od posortowania malejąco zadań względem sumy czasów wykonywania operacji wewnątrz każdego zadania. Można użyć odwróconej kolejki priorytetowej lub stworzyć zbiór $\mathcal{W}$ oraz następnie go posortować. Teraz w pętli należy wziąć zadanie j^* o największej sumie operacji oraz wstawić je na wszystkich kolejnych, możliwych pozycjach w permutacji częściowej π' (dla zadania k -tego w kolejności będzie k wstawień) oraz wybrać tę pozycję, dla której $C_{\max}$ jest najmniejszy. Następnie przejść do kolejnego zadania. Schemat metody pokazano w Algorytmie 1.

Algorithm 1 NEH Algorithm

```

1:  $\mathcal{N} \leftarrow \mathcal{J}$ 
2:  $k \leftarrow 1$ 
3: procedure NEH( $\mathcal{N}$ )
4:   for each  $j \in \mathcal{N}$  do
5:      $\omega_j \leftarrow \sum_{i \in \mathcal{M}} p_{ij}$ 
6:      $\mathcal{W}.\text{PUSH}(\omega_j, j)$ 
7:   end for
8:   while  $\mathcal{W} \neq \emptyset$  do
9:      $j^* \leftarrow \underset{j \in \mathcal{W}}{\operatorname{argmax}} \omega_j$ 
10:    for  $l \leftarrow 1, l \leq k$  do
11:      if  $\pi'.\text{INSERT}(j^*, l).C_{\max} < \pi^*.C_{\max}$  then
12:         $\pi^* \leftarrow \pi'.\text{INSERT}(j^*, l)$ 
13:      end if
14:       $l \leftarrow l + 1$ 
15:    end for
16:     $\pi' \leftarrow \pi^*$ 
17:     $\mathcal{W} \leftarrow \mathcal{W} \setminus \{j^*\}$ 
18:     $k \leftarrow k + 1$ 
19:  end while
20: end procedure

```

NEH+

Algorytm NEH można poprawić poprzez dodanie drugiej fazy wstawień dla zadań już przydzielonych. Należy wybrać zadanie x według jednej z czterech reguł pomijając zadanie, które właśnie wstawiliśmy:

1. Zadanie zawierające najdłuższą operację na ścieżce krytycznej.
2. Zadanie zawierające największą sumę operacji wchodzących w ścieżkę krytyczną.
3. Zadanie zawierające największą liczbę operacji wchodzących w ścieżkę krytyczną
4. Zadanie, którego usunięcie spowoduje największe zmniejszenie wartości $C_{\max}$.

Algorytm INSA

Algorytm INSA dla problemu $J||C_{\max}$ działa analogicznie jak algorytm NEH dla $FP||C_{\max}$. Największą różnicą jest to, że należy dokonać wstawień operacji niezależnie na wszystkich pozycjach na odpowiedniej maszynie.

5 Sposób generowania instancji

Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla każdego $j \in \mathcal{J}$:
 - (a) $o_j \leftarrow \text{nextInt}(1, \lfloor m \cdot 1, 2 \rfloor)$.
 - (b) Dla każdego $k \in \{1, 2, \dots, o_j\} : p_{k,j} \leftarrow \text{nextInt}(1, 29)$.
3. Dla każdego $j \in \mathcal{J}$:
 - (a) Dla każdego $k \in \{1, 2, \dots, o_j\} : \mu_{k,j} \leftarrow \text{nextInt}(1, m)$.